

A 1.5B model that writes SQL as well as a 7B

What I learned building, measuring and serving it. And three numbers I got wrong on the way.

71.5%

EXECUTION ACCURACY

2,147

HELD-OUT QUESTIONS

1.5B

PARAMETERS

Full technical write-up in progress

Ask a database a question in English

You have a database. Someone asks a question in plain English. Something has to turn that into SQL that actually runs and returns the right rows.

That is easy on a database you were trained on. This project does it on databases the model has never seen, at 1.5 billion parameters, which is small enough to serve without a GPU budget behind it.

2,147

HELD-OUT QUESTIONS

206

UNSEEN DATABASES

1.5B

PARAMETERS

Measure by running the SQL, not by reading it

The obvious way to grade generated SQL is to compare its text against a reference query. That marks two identical answers wrong for choosing a different join order.

Instead: run both queries against the real database, and compare the rows that come back. Two queries that look nothing alike and return the same answer are both correct.

That single component then does four jobs:

- the benchmark metric
- the reward signal for reinforcement learning
- the filter that verifies synthetic training data
- the vote at inference time

End to end, not a notebook

- ▶ **Qwen2.5-Coder-1.5B**
the base model
- ▶ **Supervised fine-tuning**
on Spider train, full schemas
- ▶ **GRPO**
reinforcement learning, reward is whether the query returns the right rows
- ▶ **Execution voting**
sample k answers, run them all, take the majority
- ▶ **FastAPI service**
returns SQL, rows, a calibrated confidence and a timing breakdown

338

TESTS

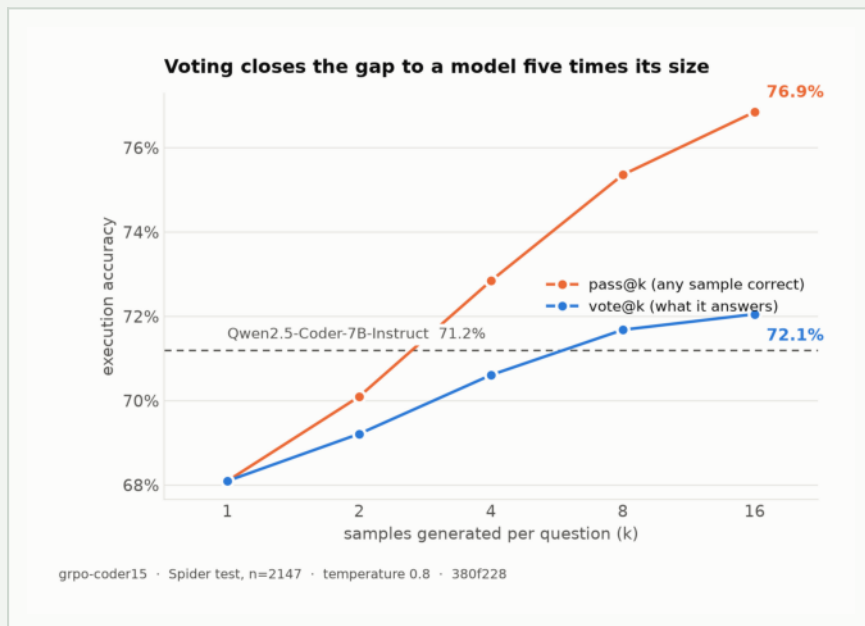
11

GPU-HOURS MEASURING

~\$14

TOTAL COMPUTE

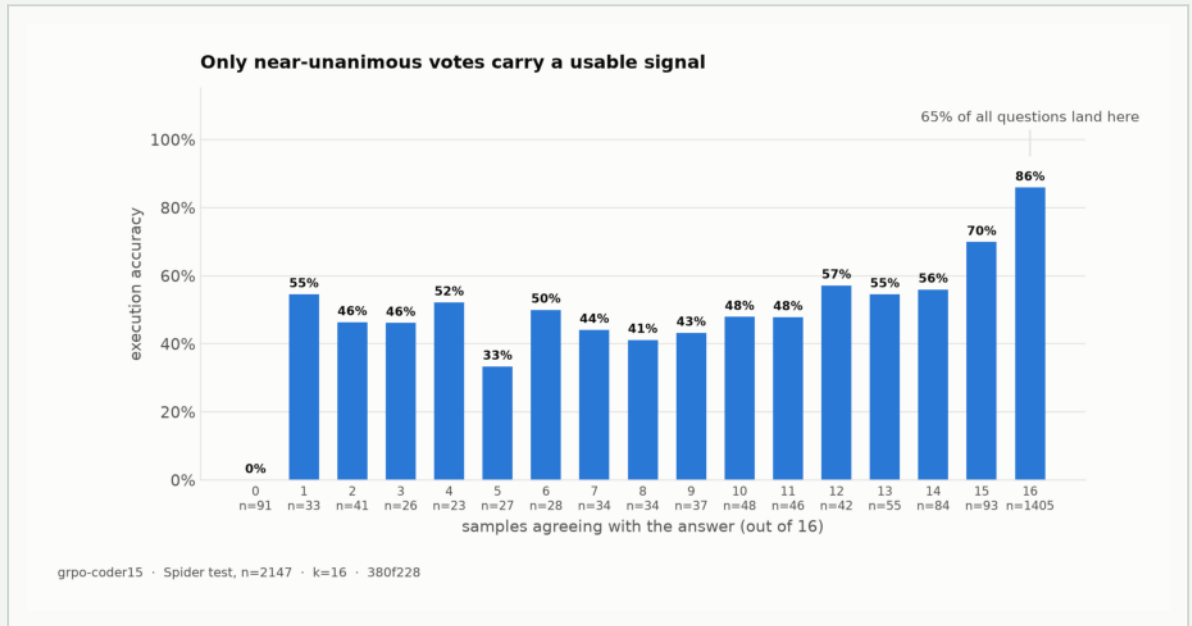
71.5%, level with a model five times larger



Sample eight answers, run all eight, reply with whatever the majority returned. 68.1% to 71.5%. A 7B scores 71.2%. Measured cost: 2.1x median latency, not 8x.

But it saturates: +1.2, +1.4, +1.1, then +0.4 per doubling. The orange ceiling keeps climbing, so the gap between what the model produces and what it picks widens to 4.8 points. The bottleneck is no longer generation. It is selection.

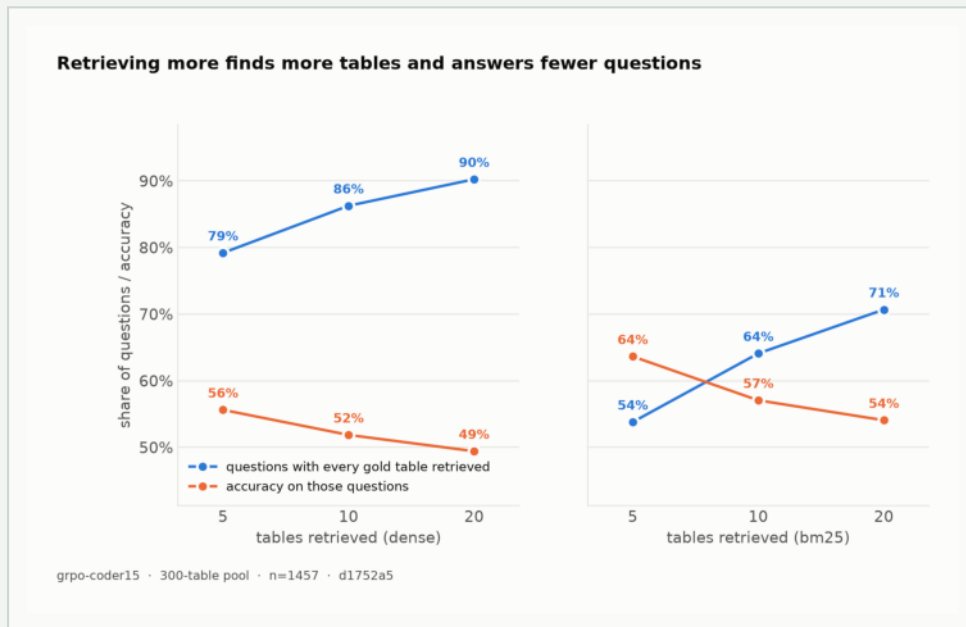
The model already knows when it is wrong



When all sixteen samples agree it is right 86% of the time, over 65% of questions. When they disagree it is close to a coin flip.

No extra training, no extra infrastructure. You are already generating the samples in order to vote. The ragged middle is honest: those buckets hold 23 to 84 questions each, so they are noise, not signal.

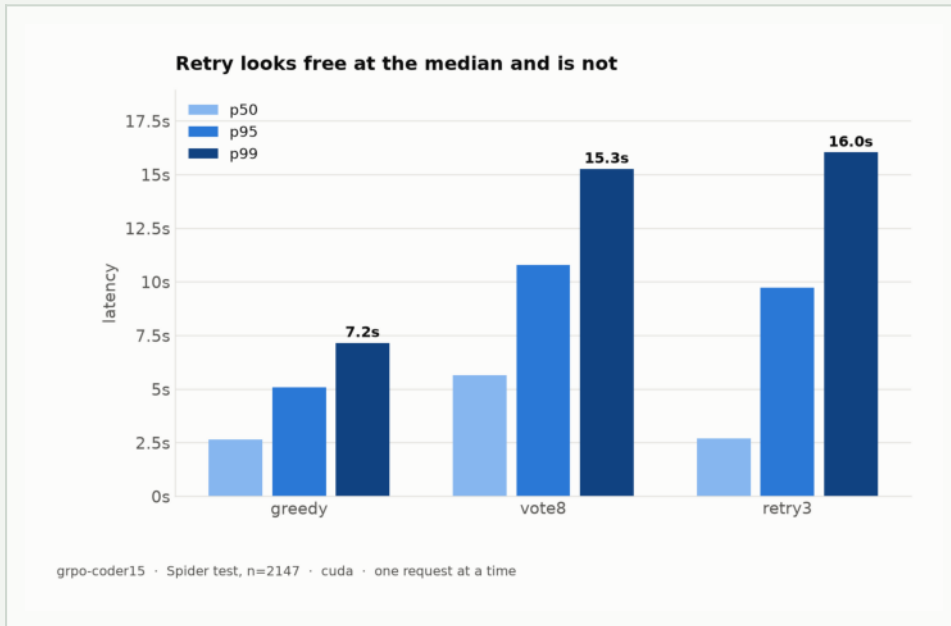
A flat number hiding two opposing forces



Hand the model a 300-table pool instead of the right database and it must retrieve first. Accuracy read 44.3%, 45.2%, 44.8% across a fourfold change in k. Flat enough to conclude k does not matter.

Split by whether retrieval found every needed table: coverage climbs while accuracy on those same questions falls, in both retrievers. They cancel. Irrelevant context costs 2.5x what missing context does.

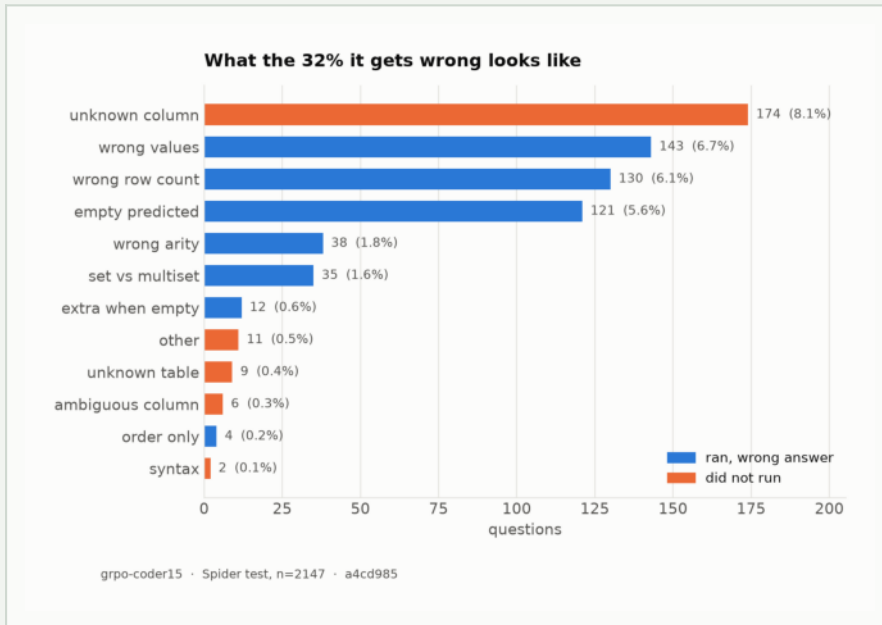
Accuracy reported next to its price



Almost nobody publishes latency beside accuracy, which is how a technique that looks free in a benchmark becomes expensive in production.

Retry's median is indistinguishable from greedy: most requests never enter the loop. All of its cost sits in the tail, and its mean hides that from both ends at once.

The largest failure is not reasoning



Every failure on the held-out split, by kind. Orange never ran. Blue ran cleanly and returned the wrong rows.

The biggest single one is the model inventing a column name that does not exist: 8.1%, larger than any wrong-answer category, after RL already cut it by 29%. The next fix belongs in how the schema is shown, not in how the model reasons.

Three numbers I got wrong

01 I reported a p99 latency of 28 seconds.

It was measured over 100 requests, and a p99 over 100 samples is the single slowest request in the sample. Over the full 2,147 it is 16 seconds, and the ranking I had built on it does not hold.

02 I wrote that retrieving fewer tables would help.

It was a guess. I ran it at 5, 10 and 20 tables across two retrievers. It does not help. The claim is withdrawn.

03 I carried a 9-point cost for missing tables.

Measured properly it is 6.7. The other half of that decomposition reproduced exactly, which is how I knew which number had drifted.

All three sit in the write-up beside the corrected versions. The wrong ones are the more useful half.

What I would build next

- ▶ **A learned verifier.**

Majority voting leaves 4.8 points on the table at $k=16$, and two hand-built tiebreak rules recovered $+0.1$ and -0.1 . This needs a model, not a heuristic.

- ▶ **Training on cluttered schemas.**

Distraction costs 2.5x what missing tables do, so the fix belongs in training rather than in the retriever.

- ▶ **vLLM in the serving path.**

A 2.7 second median is not fast. Nothing about the current stack is optimised for inference.

I am writing this up properly, with the methods and the numbers that did not make it here.

Follow if you want it when it lands.